

CENTRO UNIVERSITÁRIO DE VIÇOSA
ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS



Projeto Arquitetural

Membros:

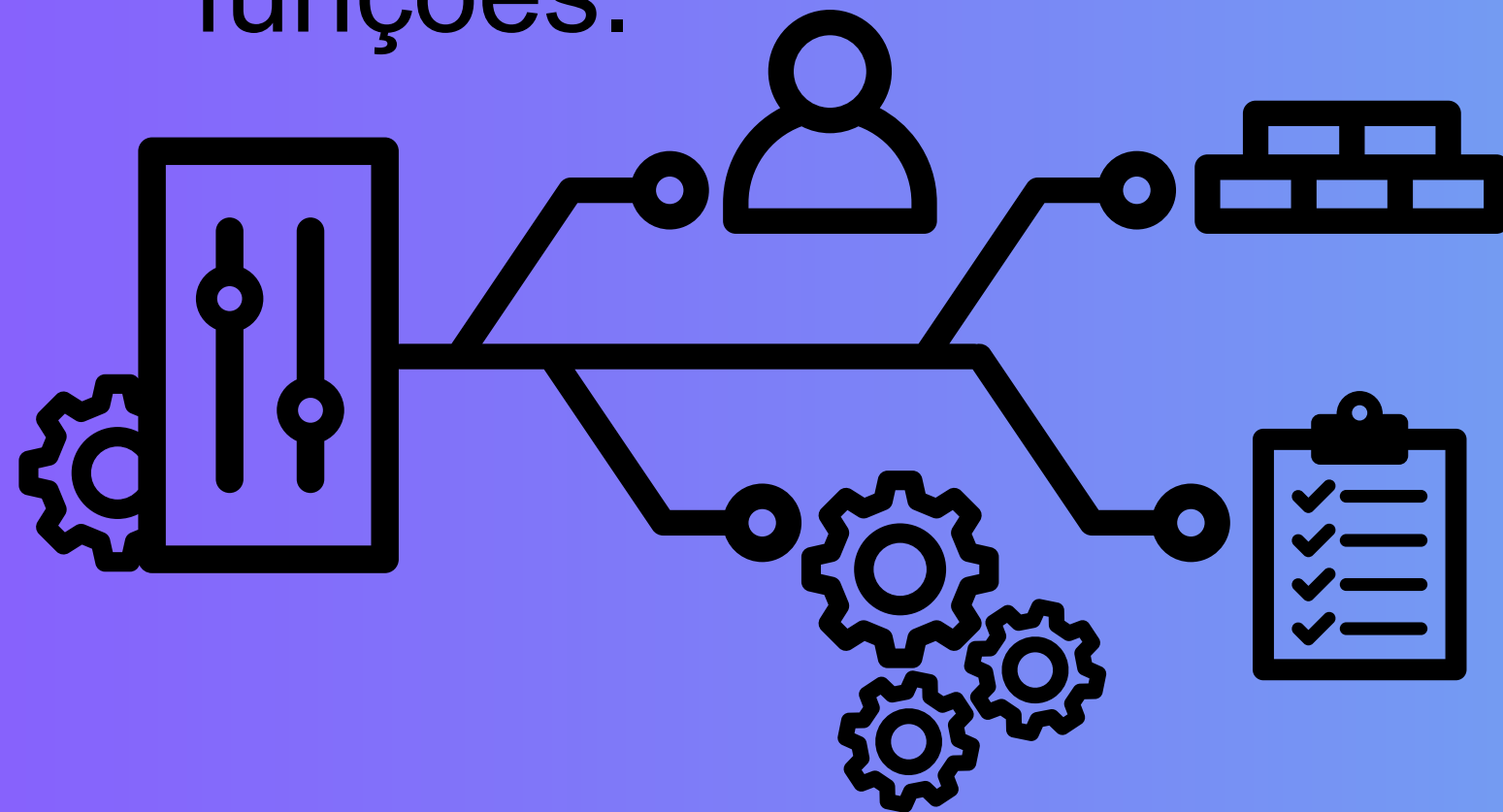
Pedro Moreira

Sumario

- Introdução
- Conhecimento de projeto
- Heurísticas de projeto
- Conceito de SOLID
- Padrões
- Anti-Padrões
- Detecção de padrões e anti-padrões

Introdução

- Decomposição da Solução de Software:
 - Descrição inicial dos elementos do sistema.
 - Visão detalhada da integração das características e funções.



Introdução

- Desafios do Projeto de Software:
 - Dificuldade em criar software reutilizável e flexível.
 - Importância de evitar reestruturação constante.



Introdução

- Elementos do Projeto de Software:
 - Heurísticas, padrões e anti-padrões como orientações-chave.



Introdução

- Exemplos de heurísticas
 - ☒ Todo módulo deve se comunicar com o mínimo possível de outros módulos.
 - Toda informação a respeito de um módulo deve ser privativa do mesmo.
 - Classes derivadas devem conhecer as suas classes base, por definição, mas classes base não devem ter qualquer tipo de conhecimento a respeito de suas classes derivadas.

Conhecimento de Projeto

- Reconhecimento da Importância da Representação de Conhecimento
 - Projetistas de sistemas reconhecem a necessidade de representar e explorar o conhecimento adquirido.

Conhecimento de Projeto

- Utilização de Boas Práticas em Projetos de Software
 - Reconhecimento e utilização de **práticas bem-sucedidas e mal-sucedidas** em projetos anteriores.
 - Inclui **heurísticas de projeto, padrões e anti-padrões** como fontes de conhecimento.

Conhecimento de Projeto

- Reconhecimento da Importância da Representação de Conhecimento
 - Projetistas de sistemas reconhecem a necessidade de representar e explorar o conhecimento adquirido.



Conhecimento de Projeto

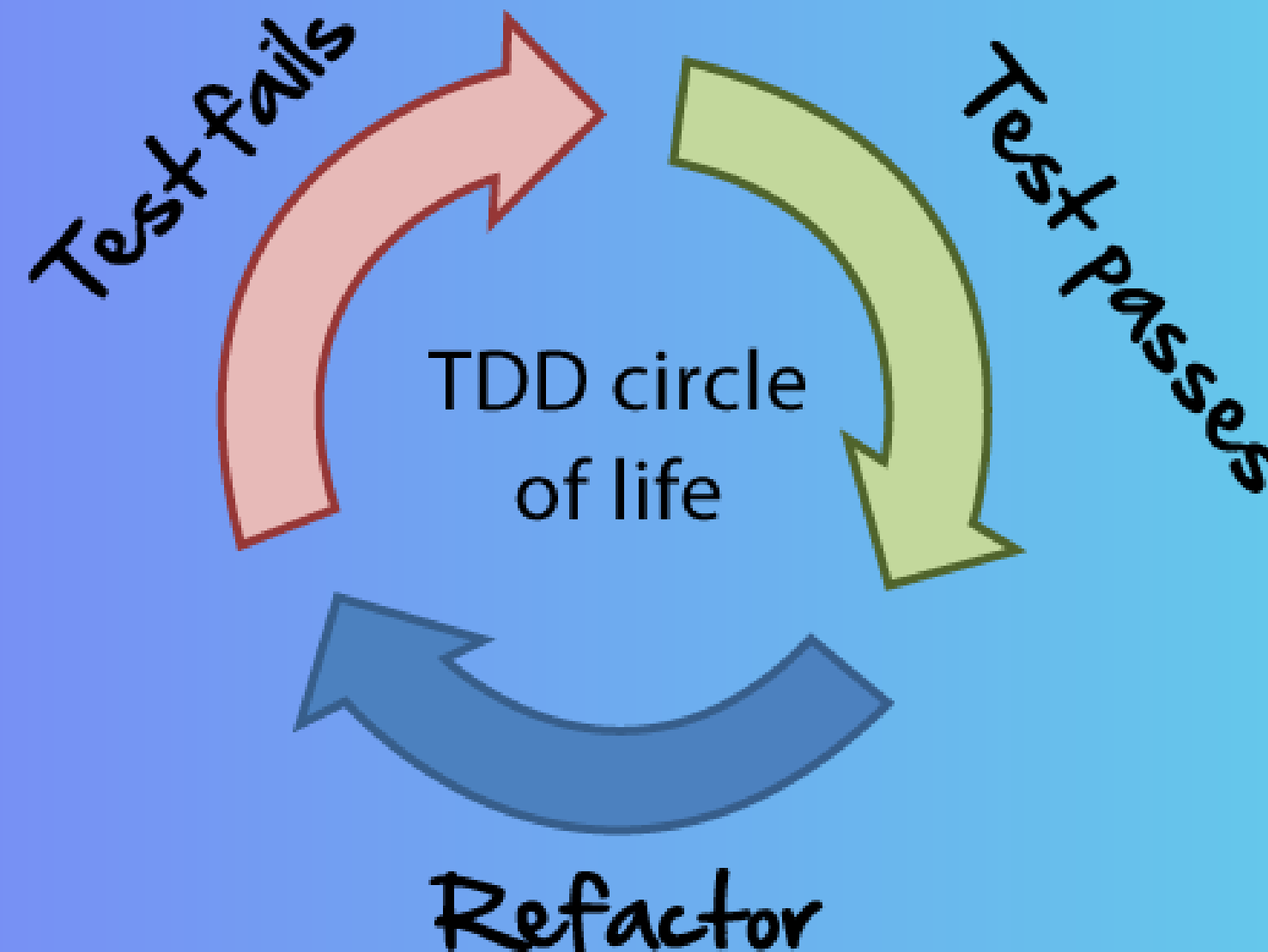
- Padrões:
 - Soluções comprovadas para problemas recorrentes de projeto de software.
 - Geram documentação
 - Economizam tempo e custo para o projeto

Conhecimento de Projeto

- Anti-Padrões:
 - Práticas que levam a resultados indesejados ou problemas em projetos de software
 - Manias e aprendizados não condizentes com o desenvolvimento do projeto.
 - Resultam em mais gastos e mais tempo de desenvolvimento.

Conhecimento de Projeto

- Importância da Experiência Prática:
 - Aprendizado com situações de sucesso e fracasso em projetos anteriores.
 - Utilização desse conhecimento para aprimorar os futuros projetos de software.



Heurísticas de Projeto

- Conceito de Heurística de Projeto:
 - Diretriz baseada em conhecimento e experiência para orientar decisões de projeto.
 - Importância na elaboração de soluções eficazes durante o desenvolvimento.



Heurísticas de Projeto

- Flexibilidade das Heurísticas:
 - Não devem ser consideradas como regras inflexíveis.
 - Violá-las pode indicar problemas nas decisões de projeto.
- Dispersão e Dificuldades na Identificação:
 - Heurísticas estão dispersas em várias fontes e descritas de maneira não uniforme.
 - Difícil reconhecimento devido à sua descrição implícita em muitos casos.

Heurísticas de Projeto

- Exemplos de Heurísticas:
 - Minimizar comunicação entre módulos.
 - Manter informações privativas de cada módulo.
 - Classes derivadas devem conhecer suas classes base, mas não o contrário.

Heurísticas de Projeto

- Proposta de Representação de Heurísticas:
 - Título, descrição, motivação e fontes bibliográficas.
 - Relacionamento com padrões e anti-padrões para ampliar compreensão.
- Utilização e Aplicação das Heurísticas:
 - Importância de integrar heurísticas ao processo de projeto de software.
 - Ferramenta valiosa para orientar tomada de decisões e evitar problemas.

Conceito de SOLID

- S - Princípio da Responsabilidade Única (Single Responsibility Principle)
 - Definição: Um objeto deve ter apenas uma razão para mudar.
 - Palavras-chave: Coesão, separação de preocupações, foco em uma única responsabilidade.
- O - Princípio do Aberto/Fechado (Open/Closed Principle)
 - Definição: Entidades de software devem ser abertas para extensão, mas fechadas para modificação.
 - Palavras-chave: Extensibilidade, encapsulamento, uso de interfaces.

- L - Princípio da Substituição de Liskov (Liskov Substitution Principle)
 - Definição: Objetos de uma classe derivada devem poder substituir objetos de sua classe base sem interromper o comportamento do programa.
 - Palavras-chave: Herança, polimorfismo, contratos.
- I - Princípio da Segregação de Interfaces (Interface Segregation Principle)
 - Definição: Muitas interfaces específicas são melhores do que uma única interface geral.
 - Palavras-chave: Interfaces coesas, clientes dependentes apenas do que precisam.

Conceito de SOLID

- D - Princípio da Inversão de Dependência (Dependency Inversion Principle)
 - Definição: Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.
 - Palavras-chave: Desacoplamento, injeção de dependência, abstrações.

Padrões

- Definição de Padrão:
 - Reutilização da essência de uma solução para problemas similares.
 - Resolve problemas recorrentes em um contexto específico.
- Características dos Padrões:
 - Fornecem uma solução comprovadamente funcional.
 - Informam os resultados e compromissos da sua aplicação.
 - Permitem adaptação para variantes do problema.

Padrões

- Diferença entre Padrões e Heurísticas:
 - Padrões são descritos explicitamente e organizadamente na literatura.
 - Heurísticas podem ser mais implícitas e dispersas.
- Descrição de um Padrão:
 - Nome do padrão, problema, contexto, solução, consequências, uso conhecido e padrões relacionados.



Padrões

- Complexidade dos Sistemas de Padrões:
 - Maior número de padrões dificulta entendimento e uso.
 - Organização eficiente é crucial para facilitar a escolha dos padrões mais adequados.
- Foco em Padrões Arquiteturais e de Projeto:
 - Padrões arquiteturais: enfatizam a organização estrutural do software.
 - Padrões de projeto: refinam e estendem a arquitetura fundamental do sistema.

Anti-Padrões

- Origem dos Anti-padrões:
 - Resultam da falta de conhecimento de soluções adequadas ou da aplicação incorreta de padrões.
 - Descrevem soluções inadequadas que levam a situações ruins ou como sair delas para alcançar uma solução melhor.
- Importância da Identificação de Anti-padrões:
 - Presença de "bons" padrões em um sistema não garante o sucesso.
 - Anti-padrões identificam construções inadequadas em sistemas malsucedidos que devem ser evitadas.

Anti-Padrões

- Estrutura de Descrição de Anti-padrões (segundo Brown):
 - Nome do anti-padrão.
 - Forma geral do problema.
 - Sintomas e consequências da sua ocorrência.
 - Causas típicas que levam ao anti-padrão.
 - Variações ou contextos em que o anti-padrão pode surgir.
 - Nova solução sugerida para evitar o anti-padrão.
 - Exceções conhecidas e soluções relacionadas para compreender melhor o problema.

Detecção de Padrões e Anti-Padrões

- Objetivo da Detecção:
 - Suporte para reestruturação de sistemas legados e avaliação de sistemas em desenvolvimento.
 - Identificação e substituição de construções problemáticas (anti-padrões) que comprometem futuras expansões ou modificações.
- Importância da Detecção de Padrões:
 - Compreensão do projeto em um nível mais abstrato.
 - Avaliação da utilização de padrões no projeto.

Detecção de Padrões e Anti-Padrões

- Desafios da Identificação Manual:
 - Sistemas legados de médio/grande porte são difíceis de analisar manualmente.
 - Apenas o código fonte muitas vezes é a fonte de informação disponível e confiável.
 - Limitações na detecção de problemas devido à análise manual restrita e custos envolvidos.

Detecção de Padrões e Anti-Padrões

- Benefícios de uma Base de Construções de Projeto:
 - Fornecimento de suporte valioso na identificação das construções adequadas.
 - Ajuda os desenvolvedores a saber que tipos de construções procurar no código.

Detecção de Padrões e Anti-Padrões

- Mecanismo de Detecção:
 - Atua sobre um modelo orientado a objetos extraído por ferramentas de engenharia reversa ou ambiente de modelagem.
 - Gera uma representação Prolog baseada em um meta-modelo específico.
 - Aplica consultas Prolog à representação do modelo para identificar padrões e anti-padrões.

Detecção de Padrões e Anti-Padrões

- Estrutura de Descrição de Anti-padrões (segundo Brown):
 - Nome do anti-padrão.
 - Forma geral do problema.
 - Sintomas e consequências da sua ocorrência.
 - Causas típicas que levam ao anti-padrão.
 - Variações ou contextos em que o anti-padrão pode surgir.
 - Nova solução sugerida para evitar o anti-padrão.
 - Exceções conhecidas e soluções relacionadas para compreender melhor o problema.

Deteção de Padrões e Anti-Padrões

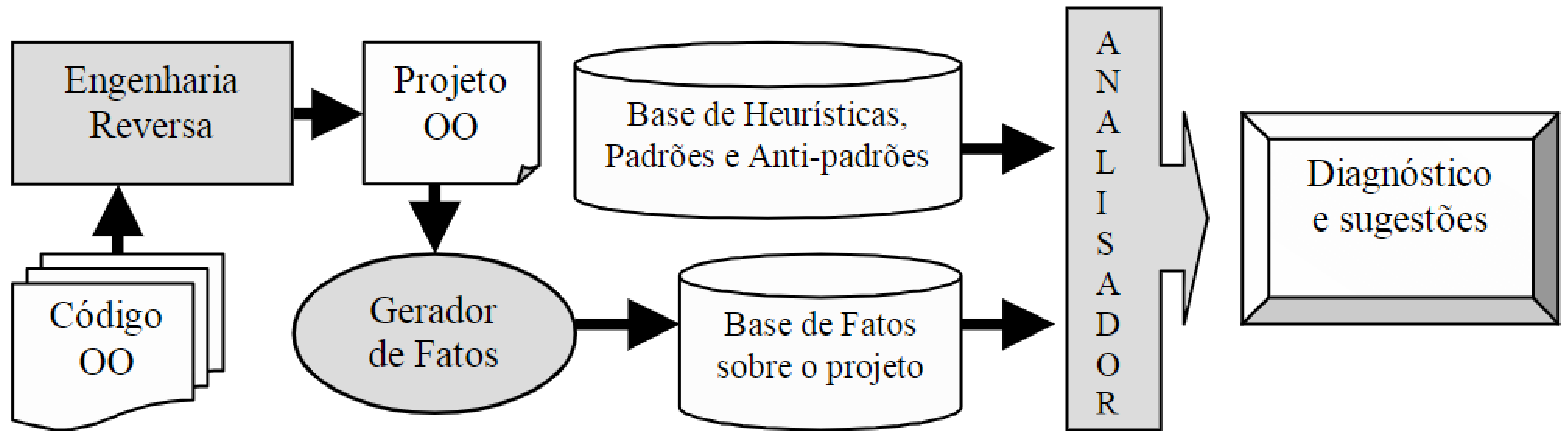


Figura 1: Visão geral da estrutura do mecanismo de detecção
(Fonte: Suporte a Padrões no Projeto de Software)

Detecção de Padrões e Anti-Padrões

- Fluxo da Detecção:
 - Extração do modelo usando ferramenta de engenharia reversa.
 - Geração automática de representação Prolog do modelo.
 - Seleção de padrões, anti-padrões ou heurísticas a serem detectados pelo usuário.
- Consulta à Base de Conhecimento:
 - Consulta à base de heurísticas, padrões e anti-padrões conforme seleção do usuário.
 - Utilização de máquina de inferência Prolog para análise dos elementos do modelo.

Detecção de Padrões e Anti-Padrões

- Relacionamento entre Padrões, Anti-padrões e Heurísticas:
 - Padrões podem ser solução para anti-padrões ou reforçar heurísticas.
 - Anti-padrões podem violar determinadas heurísticas.
 - Importância do registro desses relacionamentos pelo usuário.

Detecção de Padrões e Anti-Padrões

- Detectabilidade no Modelo Extraído:
 - Padrões e anti-padrões são detectáveis diretamente no modelo em cláusulas Prolog.
 - Heurísticas não têm representação formal, mas podem ser avaliadas indiretamente por suas relações com padrões e anti-padrões.
- Análise Indireta das Heurísticas:
 - Uma heurística pode ser considerada violada se seus anti-padrões relacionados forem detectados no modelo.

Referencias

- Dantas, Alexandre, et al. "Suporte a padrões no projeto de software." Anais do XVI Simpósio Brasileiro de Engenharia de Software. SBC, 2002.
- Martins, José Carlos Cordeiro. Técnicas para gerenciamento de projetos de software. Brasport, 2007.
- BUSCHMANN, F., MEUNIER R., ROHNERT, H., et alli. Pattern-Oriented Software Architecture, A System of Patterns. John Wiley & Sons, 1996.
- BROWN, W., MALVEAU, R. McCORNICK III, MOWBRAY, T., et alli, Anti-patterns – Refactoring Software, Architectures, and Projects in Crisis, Wiley Computer Publishing, 1998.